

STENCIL

Runtime Library & Dependency Injection Technical Specification

Version 1.0 · April 2026 · Confidential

1. Overview and Purpose

This document covers three interconnected concerns that are outside the main Stencil architecture spec: the two-repository model, the language-specific runtime libraries, and the dependency injection system used to wire generated code with developer-written hooks.

Scope

This spec covers: (1) how the CLI repo and runtime library repos relate, (2) what the stencil-go runtime library contains and why, (3) how the DI container works, (4) how generated code is wired with developer hooks, (5) how main.go and config are handled as developer-editable files.

The three artifacts

Artifact	Repo	Used at
stencil CLI	github.com/stencil-run/stencil	Build time — reads YAML, generates code
stencil-go library	github.com/stencil-run/stencil-go	Runtime — imported by generated Go services
stencil-java library	github.com/stencil-run/stencil-java	Runtime — imported by generated Java services (Phase 2)

Why separate repos

The CLI is a build-time tool written in Go — its language is an implementation detail. The runtime library is a production dependency whose language must match the generated code. A Java service cannot import a Go module. Each library also evolves at its own pace — a Java annotation change should not force a CLI release.

2. Repository Structure

2.1 CLI Repository — github.com/stencil-run/stencil

The CLI binary. Written in Go. Generates code for any target language. Contains no runtime code — nothing in this repo runs inside a generated service.

```
github.com/stencil-run/stencil/
  cmd/stencil/           ← binary entrypoint
  internal/              ← all generator internals (not importable externally)
    spec/                ← parser, validator, resolver
    diff/                ← planner, DAG, lock file
    generator/           ← interface, registry, orchestrator
    generators/
      go/                ← generates Go source files
      java/              ← generates Java source files (Phase 2)
      shared/            ← language-agnostic (migration SQL)
    imports/             ← import collector, hierarchy validator
    template/            ← engine, funcmap, postprocess
    emitter/             ← file writer, chmod, lock file
    version/             ← CLI version + compat table
  templates/
    go/                  ← *.go.tmpl files (embedded in binary)
    java/                ← *.java.tmpl files
    sql/                 ← *.sql.tmpl files
  testdata/
  e2e/
  go.mod                 ← only CLI dependencies, no runtime deps
  Makefile
```

2.2 Go Runtime Repository — github.com/stencil-run/stencil-go

The Go runtime library. Pure Go. No CLI code, no YAML parsing. Everything in this repo runs inside generated Go services in production.

```
github.com/stencil-run/stencil-go/
  container/             ← DI container
    container.go
    container_test.go
  errors/                ← domain errors, validation errors, HTTP mapping
    errors.go
    errors_test.go
  handler/              ← request binding, response writing
    handler.go
    handler_test.go
  pagination/            ← cursor + offset page types and helpers
    pagination.go
    pagination_test.go
  middleware/            ← gin middleware: auth, logging, recovery, CORS, rate
  limit
    middleware.go
    middleware_test.go
  cache/                ← BaseCache — Redis read/write/delete helpers
    cache.go
```

```

    cache_test.go
tx/                                ← RunTx — DB transaction wrapper
  tx.go
  tx_test.go
messaging/                         ← BaseProducer, BaseConsumer, Message
  messaging.go
  messaging_test.go
observability/                    ← Metrics, StartSpan, DBInstrument
  observability.go
  observability_test.go
go.mod                             ← runtime dependencies only
                                   (gin, redis, kafka, prometheus, otel...)
README.md

```

2.3 Java Runtime Repository — github.com/stencil-run/stencil-java

Phase 2. The Java runtime library. Published as a Maven artifact. Uses standard Spring Boot conventions so generated Java code looks idiomatic.

```

github.com/stencil-run/stencil-java/
src/main/java/run/stencil/
  container/                      ← Spring-aware DI helpers
  errors/                        ← StencilException, ErrorResponse
  pagination/                   ← Page<T>, CursorPage<T>
  middleware/                   ← Spring filters (auth, logging, rate limit)
  cache/                        ← BaseCache for Redis
  tx/                           ← transaction helpers
  messaging/                    ← Kafka base producer/consumer
pom.xml                          ← Maven: run.stencil:stencil-java:VERSION
README.md

```

3. Versioning Contract

The CLI and runtime libraries version independently. The CLI embeds a compatibility table that maps its version to the minimum required library version per language. The GoMod generator reads this table and pins the correct version in the generated go.mod.

3.1 Compatibility Table

Lives inside the CLI binary. Updated when a library version bump is required. The rule: library gets a new version only when its public API changes. A CLI-only change (new DSL keyword, new template feature) does not require a library bump.

```
// internal/version/compat.go

var CLIVersion = "dev"    // set by ldflags at build time

// CompatTable: CLI version → minimum library version per language.
var CompatTable = map[string]map[string]string{
    "v0.1.0": {"go": "v0.1.0", "java": "v0.1.0"},
    "v0.2.0": {"go": "v0.2.0", "java": "v0.1.2"},
    // go library had a breaking change in v0.2.0
    // java library only needed a patch so stays v0.1.2
    "v0.3.0": {"go": "v0.2.1", "java": "v0.2.0"},
    // go library had a patch (v0.2.1) — no CLI bump needed for that
    // java library had a minor bump (v0.2.0)
}

func RequiredLibVersion(cliVersion, lang string) string {
    if versions, ok := CompatTable[cliVersion]; ok {
        if v, ok := versions[lang]; ok { return v }
    }
    return "latest"
}
```

3.2 GoMod Generator Pins the Version

```
// internal/generators/go/gomod.go

func (g *GoModGenerator) Generate(ctx GeneratorContext) ([]File, error) {
    // pin the library version that matches this CLI version
    libVersion := version.RequiredLibVersion(version.CLIVersion, "go")

    deps := []Dependency{
        {Path: "github.com/gin-gonic/gin",          Version: "v1.9.1"},
        {Path: "github.com/stencil-run/stencil-go",  Version: libVersion},
        // other deps derived from spec touches...
    }
    // add deps only for infra declared in spec
    if ctx.Spec.Cache != nil {
```

```
        deps = append(deps, Dependency{Path: "github.com/redis/go-redis/v9",
Version: "v9.3.0"})
    }
    if ctx.Spec.Messaging != nil {
        deps = append(deps, Dependency{Path: "github.com/segmentio/kafka-go",
Version: "v0.4.47"})
    }
    return []File{{Path: "go.mod", Content: renderGoMod(ctx.Spec.Module, deps)}},
    nil
}
```

3.3 Developer Upgrade Flow

When a developer upgrades the CLI, the GoMod generator automatically updates the library version in go.mod on the next stencil update run.

Step	Action	Detail
1	Developer upgrades CLI	go install github.com/stencil-run/stencil/cmd/stencil@v0.4.0
2	Run stencil update	GoMod generator reads CompatTable for v0.4.0, writes new library version to go.mod
3	Run go mod tidy	Downloads new library version, updates go.sum
4	Compile	go build ./... — generated code compiles against new library version

4. stencil-go Runtime Library

Every package in stencil-go solves a problem that generated code has at runtime. The guiding principle: if generated code calls it, and it is not a third-party library, it belongs here.

4.1 container — Dependency Injection

The DI container. Used in main.go to wire generated components with developer-written hooks. Full design in Section 5.

```
import "github.com/stencil-run/stencil-go/container"

c := container.New()
c.Provide(NewUserRepository) // func(*sql.DB) *UserRepository
c.Provide(NewCreateUserHooks) // func(*UserRepository) *CreateUserHooks

var hooks *CreateUserHooks
c.MustResolve(&hooks)
```

4.2 errors — Domain Errors and HTTP Mapping

Every generated service and handler uses these types. `ValidationError` wraps go-playground/validator output into clean field-level messages. `DomainError` carries an HTTP status code. `WriteError` dispatches to the correct response based on error type.

```
import "github.com/stencil-run/stencil-go/errors"

// ValidationError — produced by MapValidationError
type ValidationError struct {
    Field   string `json:"field"`
    Message string `json:"message"`
    Code    string `json:"code"`
}
type ValidationErrors []ValidationError

// MapValidationError converts go-playground/validator output
func MapValidationError(err error) ValidationErrors { ... }

// DomainError — typed domain errors with HTTP status
type DomainError struct {
    Code    string `json:"code"`
    Message string `json:"message"`
    Status  int    `json:"-"`
}

func NewNotFound(resource string) *DomainError
func NewForbidden() *DomainError
func NewConflict(field string) *DomainError
func NewUnprocessable(reason string) *DomainError
```

```

func NewUnauthorized() *DomainError

// WriteError - used by every generated handler
// dispatches to correct HTTP status based on error type
func WriteError(c *gin.Context, err error) {
    switch e := err.(type) {
    case *DomainError:      c.JSON(e.Status, e)
    case ValidationErrors: c.JSON(400, gin.H{"errors": e})
    default:                c.JSON(500, gin.H{"code": "INTERNAL_ERROR"})
    }
}

```

4.3 handler — Request Binding and Response Writing

Generated handlers call Bind to parse and validate in one step. Write helpers ensure consistent response format and status codes across all endpoints.

```

import "github.com/stencil-run/stencil-go/handler"

// Bind parses JSON body + validates struct tags in one call.
// Returns typed pointer. Returns ValidationErrors if invalid.
func Bind[T any](c *gin.Context) (*T, error) {
    var req T
    if err := c.ShouldBindJSON(&req); err != nil {
        return nil, errors.NewUnprocessable(err.Error())
    }
    if err := validate.Struct(&req); err != nil {
        return nil, errors.MapValidationError(err)
    }
    return &req, nil
}

// Response writers - enforce consistent status codes
func WriteCreated(c *gin.Context, data any) { c.JSON(201, data) }
func WriteOK(c *gin.Context, data any)      { c.JSON(200, data) }
func WriteNoContent(c *gin.Context)         { c.Status(204) }

// Usage in generated handler:
func (h *UserHandler) CreateUser(c *gin.Context) {
    req, err := handler.Bind[CreateUserRequest](c)
    if err != nil { errors.WriteError(c, err); return }

    resp, err := h.svc.CreateUser(c.Request.Context(), *req)
    if err != nil { errors.WriteError(c, err); return }

    handler.WriteCreated(c, resp)
}

```

4.4 pagination — Page Types and Helpers

Generated list endpoints return these types. Cursor pagination for large datasets, offset pagination for smaller ones. Developer hooks receive these types when interacting with list operations.

```
import "github.com/stencil-run/stencil-go/pagination"

// CursorPage - for cursor-based pagination
type CursorPage[T any] struct {
    Data      []T    `json:"data"`
    NextCursor string `json:"next_cursor,omitempty"`
    HasMore    bool   `json:"has_more"`
    Limit      int    `json:"limit"`
}

// OffsetPage - for offset-based pagination
type OffsetPage[T any] struct {
    Data  []T `json:"data"`
    Total int `json:"total"`
    Page  int `json:"page"`
    Limit int `json:"limit"`
    Pages int `json:"pages"`
}

// CursorParams - parsed from query string
type CursorParams struct {
    Cursor string
    Limit  int
}

func ParseCursorParams(c *gin.Context, defaultLimit, maxLimit int) CursorParams {
    ...
}
func ParseOffsetParams(c *gin.Context, defaultLimit, maxLimit int) OffsetParams {
    ...
}

func NewCursorPage[T any](items []T, limit int, getCursor func(T) string)
CursorPage[T] {
    if len(items) <= limit {
        return CursorPage[T]{Data: items, HasMore: false, Limit: limit}
    }
    return CursorPage[T]{
        Data:      items[:limit],
        NextCursor: getCursor(items[limit-1]),
        HasMore:    true,
        Limit:      limit,
    }
}
```

4.5 middleware — Gin Middleware Functions

Generated routes.go applies these middleware. Developer can also use them in custom routes registered outside the spec.

```
import "github.com/stencil-run/stencil-go/middleware"
```

```
// RequestID - adds X-Request-ID header to every request and response
func RequestID() gin.HandlerFunc

// StructuredLogger - logs method, path, status, latency, request_id as JSON
func StructuredLogger(logger *slog.Logger) gin.HandlerFunc

// Recovery - catches panics, logs stack trace, returns 500
func Recovery(logger *slog.Logger) gin.HandlerFunc

// CORS - configurable allowed origins
func CORS(allowedOrigins []string) gin.HandlerFunc

// RateLimit - token bucket, N requests per second per IP
func RateLimit(rps int) gin.HandlerFunc

// Auth - validates JWT, extracts claims into gin context
// Downstream handlers call middleware.GetCaller(c) to get caller identity
func Auth(secret string) gin.HandlerFunc

// RequireRoles - checks caller has at least one of the required roles
// Must come after Auth middleware
func RequireRoles(roles ...string) gin.HandlerFunc

// Caller - the authenticated caller, stored in gin context by Auth middleware
type Caller struct {
    ID      string
    Role    string
    Email   string
}

func GetCaller(c *gin.Context) (*Caller, bool)
func MustGetCaller(c *gin.Context) *Caller // panics if no caller
```

4.6 cache — BaseCache

Generated cache implementations embed BaseCache. The BaseCache handles key prefixing, serialisation, and the Redis client call. Generated code adds the typed Get/Set/Delete/Invalidate methods on top.

```
import "github.com/stencil-run/stencil-go/cache"

// BaseCache - embedded by every generated cache implementation
type BaseCache struct {
    client      *redis.Client
    prefix      string
    defaultTTL  time.Duration
}

func NewBaseCache(client *redis.Client, prefix string, ttl time.Duration)
*BaseCache

// Unexported - called by generated typed methods
func (c *BaseCache) Get(ctx context.Context, key string, dest any) (bool, error)
func (c *BaseCache) Set(ctx context.Context, key string, val any, ttl
time.Duration) error
```

```
func (c *BaseCache) Delete(ctx context.Context, keys ...string) error

// Generated UserCache embeds BaseCache:
// type UserCache struct {
//     cache.BaseCache
// }
// func (c *UserCache) Get(ctx context.Context, id int64) (*UserResponse, bool,
// error) {
//     var resp UserResponse
//     hit, err := c.BaseCache.Get(ctx, fmt.Sprintf("user:%d", id), &resp)
//     return &resp, hit, err
// }
```

4.7 tx — Database Transaction Wrapper

Generated transaction orchestrators call RunTx. It handles Begin, Commit, and deferred Rollback. The developer never writes transaction management code.

```
import "github.com/stencil-run/stencil-go/tx"

// RunTx wraps fn in a DB transaction.
// Commits if fn returns nil. Rolls back if fn returns error.
// Rollback is always deferred — safe even if fn panics.
func RunTx(ctx context.Context, db *sql.DB, fn func(*sql.Tx) error) error {
    t, err := db.BeginTx(ctx, nil)
    if err != nil { return fmt.Errorf("begin tx: %w", err) }
    defer t.Rollback()
    if err := fn(t); err != nil { return err }
    return t.Commit()
}

// Usage in generated PlaceOrder transaction:
// func (t *PlaceOrderTx) Execute(ctx context.Context, p PlaceOrderParams)
// (*Result, error) {
//     var result Result
//     err := tx.RunTx(ctx, t.db, func(tx *sql.Tx) error {
//         // step 1: create order
//         // step 2: deduct inventory
//         return nil
//     })
//     return &result, err
// }
```

4.8 messaging — Base Producer and Consumer

Generated producers embed BaseProducer. Generated consumers embed BaseConsumer. Each adds typed Publish methods and typed handler interfaces on top.

```
import "github.com/stencil-run/stencil-go/messaging"

// Message — the universal envelope
type Message struct {
```

```

    Topic    string
    Key      []byte
    Payload  []byte
    Headers  map[string]string
}

// BaseProducer - embedded by generated typed producer
type BaseProducer struct {
    writer *kafka.Writer
}

func NewBaseProducer(brokers []string) *BaseProducer
func (p *BaseProducer) Publish(ctx context.Context, msg Message) error

// BaseConsumer - embedded by generated typed consumer
type BaseConsumer struct {
    reader *kafka.Reader
    dlq    *kafka.Writer
    logger *slog.Logger
}

func NewBaseConsumer(brokers []string, topic, groupID string, dlqTopic string)
*BaseConsumer

// Run - blocks, retries on error, routes to DLQ after max attempts
func (c *BaseConsumer) Run(ctx context.Context, handler func(ctx context.Context,
payload []byte) error)

```

4.9 observability — Metrics, Tracing, Logging

Generated middleware and service code calls these. The Metrics struct holds all pre-registered Prometheus counters and histograms. DBInstrument wraps any DB call with timing.

```

import "github.com/stencil-run/stencil-go/observability"

// Metrics - pre-registered Prometheus metrics
type Metrics struct {
    RequestDuration *prometheus.HistogramVec // labels: method, path, status
    RequestCount    *prometheus.CounterVec      // labels: method, path, status
    ErrorRate       *prometheus.CounterVec      // labels: method, path, code
    DBDuration      *prometheus.HistogramVec    // labels: query
    CacheHits       *prometheus.CounterVec      // labels: cache, op, hit
}

func NewMetrics(reg *prometheus.Registry) *Metrics

// StartSpan - wraps OpenTelemetry span creation
func StartSpan(ctx context.Context, name string, attrs ...attribute.KeyValue)
(context.Context, trace.Span)

// DBInstrument - wraps a DB call with span + histogram recording
func DBInstrument(ctx context.Context, m *Metrics, queryName string, fn func()
error) error {
    start := time.Now()
    ctx, span := StartSpan(ctx, "db."+queryName)
    defer span.End()
}

```

```
err := fn()  
m.DBDuration.WithLabelValues(queryName).Observe(time.Since(start).Seconds())  
return err  
}
```

5. Dependency Injection Container

The container solves the wiring problem: generated code and developer-written hooks need to be assembled at startup without either side having a direct import of the other. The container is the seam between them.

5.1 Design Decisions

Decision	Rationale
Build our own (~150 lines)	Generated graph is known at code-gen time. Container only needs to handle the user-extension seam. Uber Fx uses reflection for a problem we already solved statically.
Panic at startup on missing dep	Missing dependency = programmer error. Fail fast at startup, never silently at runtime. Same principle as Fx and dig.
Singleton by default	Each type built exactly once, cached. DB connection, Redis client, service instances — all singletons. Matches how generated wire.go works.
Reflection-based resolution	Constructor signatures inspected at runtime via reflect. Same approach as Fx internally. Allows developer to register arbitrary constructors without code generation.
Lifecycle hooks	OnStart/OnStop for clean startup and graceful shutdown. HTTP server, Kafka consumer, DB connection pool — all need controlled lifecycle.
Not Spring Boot	For Java (Phase 2), Spring Boot DI is used directly via <code>@Component/@Autowired</code> . No custom container needed — Spring handles everything.

5.2 Container Implementation

```
// github.com/stencil-run/stencil-go/container/container.go

package container

import (
    "context"
    "fmt"
    "reflect"
    "sync"
)

type Container struct {
    mu          sync.RWMutex
    providers   map[reflect.Type]reflect.Value // type → constructor func
    cache      map[reflect.Type]reflect.Value // type → built instance
    startHooks []func(context.Context) error
    stopHooks  []func(context.Context) error
}
```

```

    }

    func New() *Container {
        return &Container{
            providers: map[reflect.Type]reflect.Value{},
            cache:     map[reflect.Type]reflect.Value{},
        }
    }

    // Provide registers a constructor function.
    // The constructor can take any previously registered types as parameters.
    // Output type is the first return value of the constructor.
    // Panics if constructor is not a function or has no return values.
    func (c *Container) Provide(constructor interface{}) {
        fn := reflect.TypeOf(constructor)
        if fn.Kind() != reflect.Func {
            panic(fmt.Sprintf("container.Provide: expected func, got %T", constructor))
        }
        if fn.NumOut() == 0 {
            panic("container.Provide: constructor must return at least one value")
        }
        outType := fn.Out(0)
        c.mu.Lock()
        c.providers[outType] = reflect.ValueOf(constructor)
        c.mu.Unlock()
    }

    // MustResolve builds the target type and all its transitive dependencies.
    // target must be a pointer to the type to resolve (e.g. &myService).
    // Panics if any dependency in the graph is missing – startup-time failure.
    func (c *Container) MustResolve(target interface{}) {
        t := reflect.TypeOf(target)
        if t.Kind() != reflect.Ptr {
            panic("container.MustResolve: target must be a pointer")
        }
        val := c.resolve(t.Elem())
        reflect.ValueOf(target).Elem().Set(val)
    }

    func (c *Container) resolve(t reflect.Type) reflect.Value {
        // check cache first – each type built exactly once
        c.mu.RLock()
        if cached, ok := c.cache[t]; ok {
            c.mu.RUnlock()
            return cached
        }
        c.mu.RUnlock()

        // find constructor
        c.mu.RLock()
        constructor, ok := c.providers[t]
        c.mu.RUnlock()
        if !ok {
            panic(fmt.Sprintf("container: no provider registered for %v – add c.Provide(NewXxx) in wire.go or hooks/register.go", t))
        }

        // recursively resolve each input dependency
        fnType := constructor.Type()
        args := make([]reflect.Value, fnType.NumIn())
        for i := 0; i < fnType.NumIn(); i++ {
            args[i] = c.resolve(fnType.In(i))
        }
    }

```

```

    }

    // call the constructor
    results := constructor.Call(args)
    val := results[0]

    // cache the result
    c.mu.Lock()
    c.cache[t] = val
    c.mu.Unlock()

    return val
}

// OnStart registers a function called when Start() is invoked.
// Called in registration order.
func (c *Container) OnStart(fn func(context.Context) error) {
    c.startHooks = append(c.startHooks, fn)
}

// OnStop registers a function called when Stop() is invoked.
// Called in REVERSE registration order (like defer).
func (c *Container) OnStop(fn func(context.Context) error) {
    c.stopHooks = append(c.stopHooks, fn)
}

// Start calls all OnStart hooks in order, then blocks until ctx is cancelled.
func (c *Container) Start(ctx context.Context) error {
    for _, fn := range c.startHooks {
        if err := fn(ctx); err != nil { return err }
    }
    <-ctx.Done()
    return nil
}

// Stop calls all OnStop hooks in reverse order.
func (c *Container) Stop(ctx context.Context) error {
    var errs []error
    for i := len(c.stopHooks) - 1; i >= 0; i-- {
        if err := c.stopHooks[i](ctx); err != nil {
            errs = append(errs, err)
        }
    }
    return errors.Join(errs...)
}

```

5.3 Cycle Detection

The container detects circular dependencies by tracking the resolution stack. If a type is already being resolved when it is requested again, there is a cycle — panic with a clear message.

```

// resolve with cycle detection (added to the implementation above)

func (c *Container) resolve(t reflect.Type) reflect.Value {
    return c.resolveWithStack(t, []reflect.Type{})
}

```



```
func (c *Container) resolveWithStack(t reflect.Type, stack []reflect.Type)
reflect.Value {
    // cycle check
    for _, s := range stack {
        if s == t {
            chain := make([]string, len(stack)+1)
            for i, s := range stack { chain[i] = s.String() }
            chain[len(stack)] = t.String()
            panic(fmt.Sprintf("container: circular dependency detected: %s",
                strings.Join(chain, " → ")))
        }
    }
    // ... rest of resolve with stack passed to recursive calls
}
```

6. Wiring Model — How Generated Code Meets Developer Code

This is the complete picture of how the container connects generated infrastructure with developer-written hooks, and how both connect to main.go.

6.1 File Ownership in a Generated Project

File	Owner	When created / updated
generated/wire.go	Tool	Regenerated every stencil update — registers all generated providers
generated/config_types.go	Tool	Regenerated — typed Config struct matching spec config: block
generated/...	Tool	All other generated files — read-only, always regenerated
config/config.go	Developer	Generated ONCE by stencil generate — developer edits and owns
hooks/register.go	Developer	Generated ONCE — developer adds hook provider registrations
hooks/**/*.go	Developer	Developer writes entirely — hook implementations
main.go	Developer	Generated ONCE — developer edits to add custom startup logic

6.2 generated/wire.go — What the Tool Generates

The tool generates wire.go on every stencil update. It registers every generated component as a provider. It never registers developer hooks — those are in hooks/register.go.

```
// generated/wire.go — regenerated on every stencil update, do not edit

package generated

import (
    "orders-service/generated/tables"
    "orders-service/generated/apis"
    "orders-service/generated/cache"
    "orders-service/generated/externals"
    "orders-service/generated/messaging"
    "orders-service/generated/transactions"
```

```

    "github.com/stencil-run/stencil-go/container"
)

func Register(c *container.Container) {
    // — infra —————
    c.Provide(newDB)           // func(*Config) *sql.DB
    c.Provide(newRedis)       // func(*Config) *redis.Client
    c.Provide(newKafkaWriter) // func(*Config) *kafka.Writer

    // — tables —————
    c.Provide(tables.NewUsersRepository)
    c.Provide(tables.NewOrdersRepository)

    // — cache —————
    c.Provide(cache.NewUserCache)

    // — externals —————
    c.Provide(externals.NewStripeClient)
    c.Provide(externals.NewUserServiceClient)

    // — messaging —————
    c.Provide(messaging.NewProducer)

    // — transactions —————
    c.Provide(transactions.NewPlaceOrderTx)

    // — api services —————
    // Note: services take their hooks as a constructor param.
    // Hooks are registered by developer in hooks/register.go.
    // Container resolves them automatically.
    c.Provide(apis.NewCreateUserService)
    c.Provide(apis.NewGetUserService)
    c.Provide(apis.NewListUsersService)
    c.Provide(apis.NewPlaceOrderService)
    c.Provide(apis.NewChargeCardService)

    // — app —————
    c.Provide(NewApp)           // func(all services...) *App
    c.Provide(NewRouter)       // func(*App) *gin.Engine
}

```

6.3 hooks/register.go — Developer Registers Their Hooks

Generated ONCE by stencil generate. Developer adds one Provide call per hook implementation. The container resolves them before injecting into services.

```

// hooks/register.go - generated once, developer owns and edits

package hooks

import (
    "orders-service/generated/container"
    userhooks "orders-service/hooks/user"
    orderhooks "orders-service/hooks/order"
    payhooks   "orders-service/hooks/payment"
)

```

```

func Register(c *container.Container) {
    // User API hooks
    c.Provide(userhooks.NewCreateUserHooks)
    c.Provide(userhooks.NewGetUserHooks)
    c.Provide(userhooks.NewListUsersHooks)

    // Order API hooks
    c.Provide(orderhooks.NewPlaceOrderHooks)
    c.Provide(orderhooks.NewGetOrderHooks)

    // Payment API hooks
    c.Provide(payhooks.NewChargeCardHooks)

    // When you add a new API hook, add its provider here.
    // Run: stencil hooks scaffold <APIName> for a reminder of what to add.
}

```

6.4 How a Hook Constructor Looks

Hook constructors follow the same pattern as any other constructor. They declare their dependencies as parameters. The container injects them automatically.

```

// hooks/user/createuser.hooks.go - developer writes this

package user

import (
    "context"
    "orders-service/generated/apis"
    "orders-service/generated/cache"
    "orders-service/generated/externals"
    "golang.org/x/crypto/bcrypt"
)

type CreateUserHooks struct {
    userCache *cache.UserCache
    crmClient *externals.CRMClient
}

// NewCreateUserHooks - constructor. Container injects UserCache and CRMClient.
// No need to know how they are built - container handles that.
func NewCreateUserHooks(
    userCache *cache.UserCache,
    crmClient *externals.CRMClient,
) *apis.CreateUserHooks {
    h := &CreateUserHooks{userCache: userCache, crmClient: crmClient}
    return &apis.CreateUserHooks{
        BeforeCreateUser: h.beforeCreate,
        AfterTableUsersCreate: h.afterTableCreate,
        AfterCRMClientCreate: h.afterCRMCreate,
        BeforeResponse: h.buildResponse,
    }
}

```

```

func (h *CreateUserHooks) beforeCreate(ctx context.Context, shared
*apis.CreateUserContext) error {
    hashed, err := bcrypt.GenerateFromPassword([]byte(shared.Request.Password),
bcrypt.DefaultCost)
    if err != nil { return err }
    shared.Request.Password = string(hashed)
    return nil
}

func (h *CreateUserHooks) afterTableCreate(ctx context.Context, shared
*apis.CreateUserContext) error {
    // conditionally enable CRM sync for business accounts
    if shared.TableUsersResult.AccountType == "business" {
        shared.RunCRMClientCreate = true
    }
    return nil
}

func (h *CreateUserHooks) afterCRMCreate(ctx context.Context, shared
*apis.CreateUserContext) error {
    if shared.CRMClientError != nil {
        // non-fatal - log and clear
        shared.CRMClientError = nil
    }
    return nil
}

func (h *CreateUserHooks) buildResponse(ctx context.Context, shared
*apis.CreateUserContext) error {
    u := shared.TableUsersResult
    shared.Response = &apis.UserResponse{
        ID: u.ID, Email: u.Email,
        FullName: u.FirstName + " " + u.LastName,
    }
    return nil
}

```

6.5 main.go — Developer-Editable Entry Point

Generated ONCE by stencil generate. The developer owns it completely after that. It assembles the container, starts the app, and handles OS signals for graceful shutdown.

```

// main.go - generated once by stencil generate, developer owns after
// Modify freely: add custom startup logic, flags, custom routes, etc.

package main

import (
    "context"
    "log/slog"
    "os"
    "os/signal"
    "syscall"

    "orders-service/config"
    "orders-service/generated"

```

```

    "orders-service/generated/wire"
    "orders-service/hooks"
    "github.com/stencil-run/stencil-go/container"
)

func main() {
    logger := slog.New(slog.NewJSONHandler(os.Stdout, nil))

    // Load config - edit config/config.go to customise
    cfg := config.Load()

    // Build container
    c := container.New()

    // Register generated components (auto-maintained by stencil)
    c.Provide(func() *generated.Config { return cfg })
    wire.Register(c)

    // Register developer hook implementations
    // Edit hooks/register.go to add your hooks
    hooks.Register(c)

    // — Add custom providers here —————
    // e.g. c.Provide(custom.NewAnalyticsHandler)

    // Resolve the app - builds entire dependency graph
    var app *generated.App
    c.MustResolve(&app)

    // — Add custom routes here —————
    // e.g. app.Router.GET("/custom/route", myHandler)

    // Lifecycle: start server, stop on signal
    ctx, cancel := signal.NotifyContext(context.Background(),
        syscall.SIGINT, syscall.SIGTERM)
    defer cancel()

    c.OnStart(func(ctx context.Context) error {
        logger.Info("starting", "port", cfg.Port)
        go app.Start()
        return nil
    })
    c.OnStop(func(ctx context.Context) error {
        logger.Info("shutting down")
        return app.Stop(ctx)
    })

    if err := c.Start(ctx); err != nil {
        logger.Error("start failed", "err", err)
        os.Exit(1)
    }
    c.Stop(context.Background())
}

```

6.6 config/config.go — Developer-Editable Config

Generated ONCE. The typed Config struct lives in generated/config_types.go (always regenerated from the spec). The Load() function lives in config/config.go which the developer owns and can extend with custom env vars, feature flags, etc.

```
// generated/config_types.go - regenerated every stencil update, do not edit
// Reflects the config: block in stencil.yaml exactly
```

```
type Config struct {
    DatabaseURL string
    RedisURL    string
    JWTSecret   string
    StripeURL   string
    Port        int
    LogLevel    string
}
```

```
// config/config.go - generated once, developer owns and edits freely
```

```
package config
```

```
import (
    "fmt"
    "os"
    "strconv"
    "orders-service/generated"
)
```

```
func Load() *generated.Config {
    return &generated.Config{
        // Generated from spec config: block - do not remove these
        DatabaseURL: mustEnv("DATABASE_URL"),
        RedisURL:    mustEnv("REDIS_URL"),
        JWTSecret:   mustEnv("JWT_SECRET"),
        StripeURL:   mustEnv("STRIPE_URL"),
        Port:        envInt("PORT", 8080),
        LogLevel:    envStr("LOG_LEVEL", "info"),

        // Add custom config below this line
    }
}
```

```
func mustEnv(key string) string {
    v := os.Getenv(key)
    if v == "" { panic(fmt.Sprintf("required env var missing: %s", key)) }
    return v
}
```

```
func envInt(key string, def int) int {
    v := os.Getenv(key)
    if v == "" { return def }
    n, err := strconv.Atoi(v)
    if err != nil { return def }
    return n
}
```

```
func envStr(key, def string) string {
    v := os.Getenv(key)
    if v == "" { return def }
}
```

```
    return v  
}
```


7. Developer Code Outside the Spec

Developers may need endpoints or flows that the spec does not cover. The container makes this straightforward — custom code declares its dependencies as constructor parameters and the container injects them.

7.1 Custom Endpoint Using Generated Infra

A custom analytics endpoint that needs the `UsersRepository` and `UserCache`. The developer writes the handler, registers the constructor, and mounts the route in `main.go`.

```
// hooks/custom/analytics.go — developer writes this entirely

package custom

import (
    "net/http"
    "orders-service/generated/tables"
    "orders-service/generated/cache"
    "github.com/gin-gonic/gin"
)

type AnalyticsHandler struct {
    usersRepo *tables.UsersRepository
    userCache *cache.UserCache
}

// Constructor — container injects UsersRepository and UserCache
func NewAnalyticsHandler(
    repo *tables.UsersRepository,
    cache *cache.UserCache,
) *AnalyticsHandler {
    return &AnalyticsHandler{usersRepo: repo, userCache: cache}
}

func (h *AnalyticsHandler) UserStats(c *gin.Context) {
    count, err := h.usersRepo.CountByRole(c.Request.Context(), "admin")
    if err != nil { c.JSON(500, gin.H{"error": err.Error()}); return }
    c.JSON(200, gin.H{"admin_count": count})
}
```

```
// main.go additions — developer adds these lines

// In import block:
import "orders-service/hooks/custom"

// After wire.Register(c):
c.Provide(custom.NewAnalyticsHandler)

// After c.MustResolve(&app):
var analytics *custom.AnalyticsHandler
c.MustResolve(&analytics)
```

```
// Register custom route on the generated router
app.Router.GET("/analytics/users", analytics.UserStats)
```

7.2 Custom Complete Flow (No Generated Service Involved)

A custom report generation flow that calls the DB directly, bypasses all generated services, and has its own lifecycle. Registered and wired the same way.

```
// hooks/custom/reports.go

type ReportService struct {
    db      *sql.DB
    mailer  *externals.EmailClient
}

func NewReportService(db *sql.DB, mailer *externals.EmailClient) *ReportService {
    return &ReportService{db: db, mailer: mailer}
}

// Takes *sql.DB directly — container has it from wire.Register
// Takes *externals.EmailClient — container has it from wire.Register
// No generated service involved at all
```

7.3 The Container Resolution Sequence at Startup

This is the exact order things happen when main.go runs. Understanding this sequence is important when debugging missing dependency panics.

Step	What runs	Detail
1	config.Load()	Reads env vars. Panics immediately if required var missing.
2	container.New()	Empty container created.
3	c.Provide(cfg)	Config registered as singleton.
4	wire.Register(c)	All generated constructors registered. Nothing built yet.
5	hooks.Register(c)	Developer hook constructors registered. Nothing built yet.
6	custom registrations	Any developer custom providers registered.
7	c.MustResolve(&app)	Container walks the dependency graph, builds everything in topological order. Panics if any dep missing or cycle detected.
8	c.Start(ctx)	OnStart hooks fire. HTTP server starts. Kafka consumers start.
9	signal wait	Blocks until SIGINT or SIGTERM.

10	c.Stop(ctx)	OnStop hooks fire in reverse order. Server drains. Kafka closes.
----	-------------	--

8. The stencil hooks scaffold Command

When a developer adds a new API to their spec and runs stencil update, the new API's context, hooks interface, service, and handler are generated. But the developer still needs to create their hook implementation file and register it. The scaffold command handles the reminder and the boilerplate.

8.1 What it generates

```
$ stencil hooks scaffold CreateUser

Generated: hooks/user/createuser.hooks.go

Next: add to hooks/register.go:
      c.Provide(userhooks.NewCreateUserHooks)
```

The generated file:

```
// hooks/user/createuser.hooks.go - scaffolded, developer fills in

package user

import (
    "context"
    "orders-service/generated/apis"
    // Add your dependencies here
)

type createUserHooks struct {
    // Add your dependencies here
}

// NewCreateUserHooks - constructor, add deps as params, container injects them.
func NewCreateUserHooks(/* add deps */) *apis.CreateUserHooks {
    h := &createUserHooks{}
    return &apis.CreateUserHooks{
        // Implement the hooks you need. All are optional.

        // BeforeCreateUser: h.beforeCreate,
        // AfterTableUsersCreate: h.afterTableCreate,
        // AfterCRMClientCreate: h.afterCRM,
        // BeforeResponse: h.buildResponse,
    }
}

// func (h *createUserHooks) beforeCreate(ctx context.Context, shared
// apis.CreateUserContext) error {
//     return nil
// }
```

8.2 What the scaffold command knows

The scaffold command reads the generated hooks.go file for the requested API and generates the stub implementations for every hook point. The developer sees the full hook interface as commented-out stubs — they uncomment what they need and delete what they do not.

Developer experience goal

The developer runs `stencil hooks scaffold CreateUser` and gets a file that compiles immediately (empty hooks, no-op implementations). They uncomment hooks one at a time as they need them. The generated hooks.go is the reference — the scaffold file mirrors it exactly.

9. Java Dependency Injection (Phase 2)

For Java, Spring Boot's built-in DI is used. No custom container is needed. Generated Java code uses standard Spring annotations — `@Component`, `@Service`, `@Autowired`, `@Bean`. Spring Boot finds and wires everything automatically.

9.1 Generated Java Service

```
// generated/apis/CreateUserService.java - never edit

@Service
public class CreateUserService {

    private final UsersRepository usersRepo;
    private final StripeClient stripeClient;
    private final UserCache userCache;
    private final CreateUserHooks hooks;    // ← injected by Spring

    @Autowired
    public CreateUserService(
        UsersRepository usersRepo,
        StripeClient stripeClient,
        UserCache userCache,
        CreateUserHooks hooks
    ) {
        this.usersRepo = usersRepo;
        this.stripeClient = stripeClient;
        this.userCache = userCache;
        this.hooks = hooks;
    }
    // ...
}
```

9.2 Developer Hook Implementation

```
// hooks/user/CreateUserHooksImpl.java - developer writes this

@Component    // Spring picks this up automatically
public class CreateUserHooksImpl implements CreateUserHooks {

    private final UserCache userCache;
    private final CRMClient crmClient;

    @Autowired
    public CreateUserHooksImpl(UserCache userCache, CRMClient crmClient) {
        this.userCache = userCache;
        this.crmClient = crmClient;
    }

    @Override
    public void beforeCreateUser(CreateUserContext shared) {
        // hash password
    }
}
```

```

        shared.getRequest().setPassword(
            BCrypt.hashpw(shared.getRequest().getPassword(), BCrypt.gensalt())
        );
    }

    @Override
    public void afterTableUsersCreate(CreateUserContext shared) {
        if ("business".equals(shared.getTableUsersResult().getAccountType())) {
            shared.setRunCRMClientCreate(true);
        }
    }
}

```

Spring finds `CreateUserHooksImpl` via `@Component` scanning, sees it implements `CreateUserHooks`, and injects it wherever `CreateUserHooks` is declared as a dependency. No registration file needed — Spring handles discovery automatically.

9.3 Java main class

```

// src/main/java/run/orders/OrdersServiceApplication.java
// Generated once – developer edits freely

@SpringBootApplication
public class OrdersServiceApplication {
    public static void main(String[] args) {
        SpringApplication.run(OrdersServiceApplication.class, args);
    }
}

// Spring Boot auto-configures everything from:
// - @Service classes (generated services)
// - @Component classes (developer hooks)
// - @Repository classes (generated repositories)
// - @RestController classes (generated handlers)
// application.properties (generated from stencil.yaml config: block)

```

10. Testing the Wiring

The container wiring can be tested independently of the full runtime test. A wiring test instantiates the full container and verifies the graph compiles without missing dependencies. This is equivalent to Uber OGW's `TestFxRegistration`.

10.1 Wiring test

```
// wiring_test.go - developer writes this once

func TestWiring(t *testing.T) {
    // use test config - no real external connections
    cfg := &generated.Config{
        DatabaseURL: "postgres://localhost/testdb",
        RedisURL:    "redis://localhost:6379",
        Port:        8080,
    }

    c := container.New()
    c.Provide(func() *generated.Config { return cfg })
    wire.Register(c)
    hooks.Register(c)

    // MustResolve the app - if any dependency is missing this panics
    // recover it as a test failure instead
    defer func() {
        if r := recover(); r != nil {
            t.Fatalf("wiring failed: %v", r)
        }
    }()

    var app *generated.App
    c.MustResolve(&app)
    // if we reach here, the full graph resolved successfully
}
```

This test runs on every CI push. It catches missing hook registrations, missing providers, and circular dependencies — all at test time, not production startup time.

10.2 Container unit tests (in stencil-go repo)

```
// container/container_test.go - in stencil-go library repo

func TestResolve_Simple(t *testing.T) {
    c := New()
    c.Provide(func() *Database { return &Database{} })
    c.Provide(func(db *Database) *UserRepo { return &UserRepo{db: db} })

    var repo *UserRepo
    c.MustResolve(&repo)
}
```



```
    assert.NotNil(t, repo)
    assert.NotNil(t, repo.db)
}

func TestResolve_Singleton(t *testing.T) {
    c := New()
    calls := 0
    c.Provide(func() *Database { calls++; return &Database{} })

    var db1, db2 *Database
    c.MustResolve(&db1)
    c.MustResolve(&db2)
    assert.Equal(t, 1, calls) // built only once
    assert.Same(t, db1, db2) // same pointer
}

func TestResolve_MissingDep(t *testing.T) {
    c := New()
    c.Provide(func(db *Database) *UserRepo { return &UserRepo{db: db} })
    // no provider for *Database

    assert.Panics(t, func() {
        var repo *UserRepo
        c.MustResolve(&repo)
    })
}

func TestResolve_CycleDetection(t *testing.T) {
    c := New()
    // A depends on B, B depends on A
    c.Provide(func(b *B) *A { return &A{} })
    c.Provide(func(a *A) *B { return &B{} })

    assert.Panics(t, func() {
        var a *A
        c.MustResolve(&a)
    })
}
```